

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Moh Nabil Zarif Rofif - 5024241038

2026

1 Langkah-Langkah Percobaan

1.1 A. Persiapan Alat, Bahan dan Lingkungan Percobaan

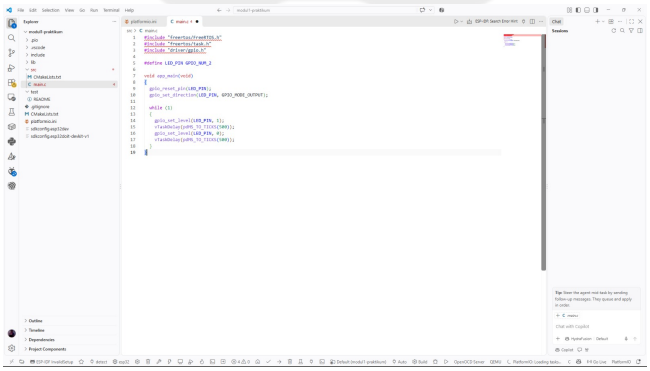
Mempersiapkan segala hal yang diperlukan, mulai dari perangkat lunak seperti VSCode, extension PlatformIO, driver untuk ST Link, STM32 dan ESP32. Perangkat keras seperti STM32 Blackpill, ST-Link V2, ESP32 DevKit, breadboard, kabel jumper, push button tactile, resistor 10k, LDR (Light Dependent Resistor) dan kabel USB Type C.

1.2 B. Percobaan 1 : Pengenalan STM32 Blackpill (Blink)

1. Konfigurasi file platformIO.ini sesuai STM32 dan pastikan IDE dan ekstensi ter-refresh.
2. Hubungkan ST-Link ke STM32 Blackpill dengan kabel jumper dan sambungkan ke komputer menggunakan USB.
3. Tulis kode blink untuk STM32 pada src/main.cpp lalu build dan upload.
4. Perhatikan LED pada board, pastikan nyala setengah detik, mati setengah detik, berulang terus.

1.3 C. Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Konfigurasi kembali file platformio.ini sesuai ESP32 Espressif dan pastikan IDE dan ekstensi ter-refresh.
2. Sambungkan ESP32 ke komputer.
3. Hapus file main.cpp dan buat file main.c yang berisi kode blink lalu build dan upload.

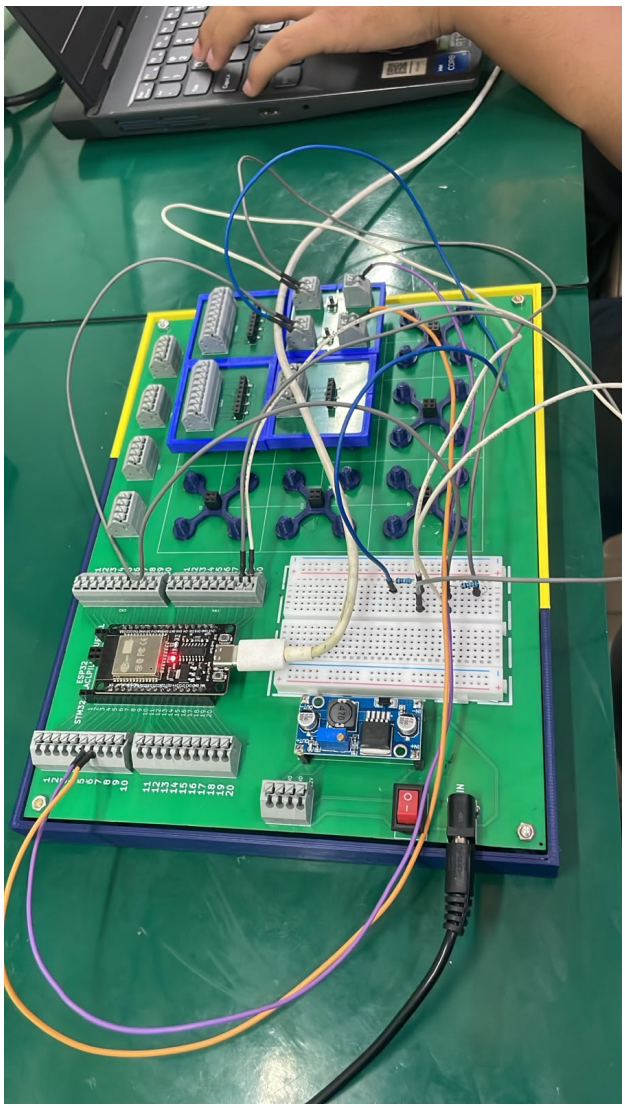


Gambar 1: Kode Blink ESP32 pada main.c

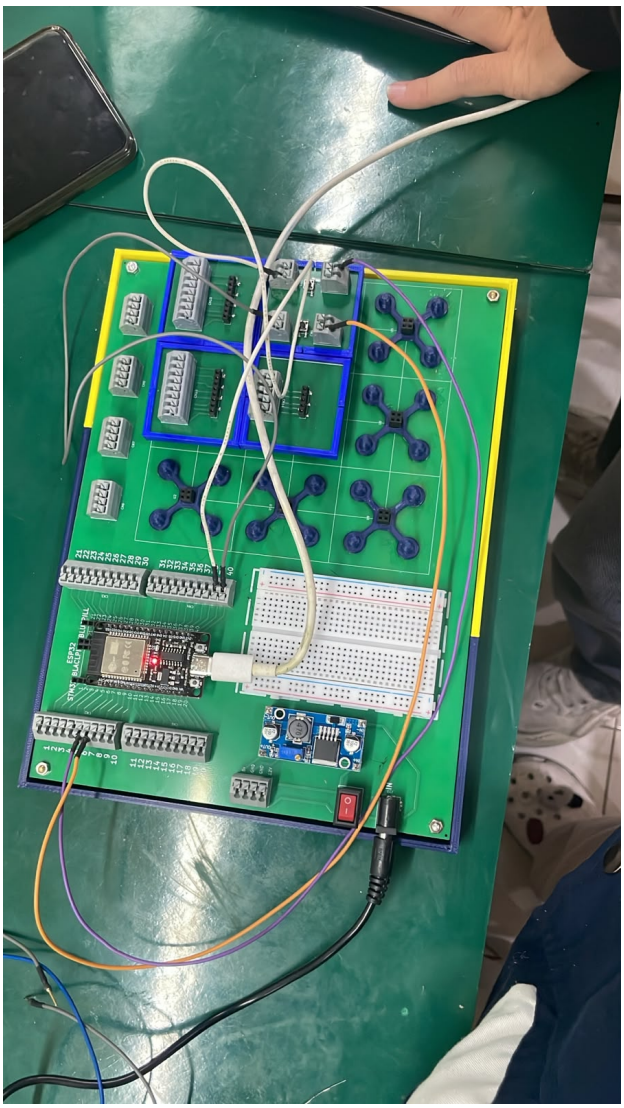
4. Mengamati dan menganalisa LED pada ESP32 apakah menyala setiap 500ms.

1.4 D. Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Rakit terlebih dahulu rangkaiannya menggunakan ESP32, resistor dan 2 button tactile yang telah disediakan lalu sambungkan ke komputer.



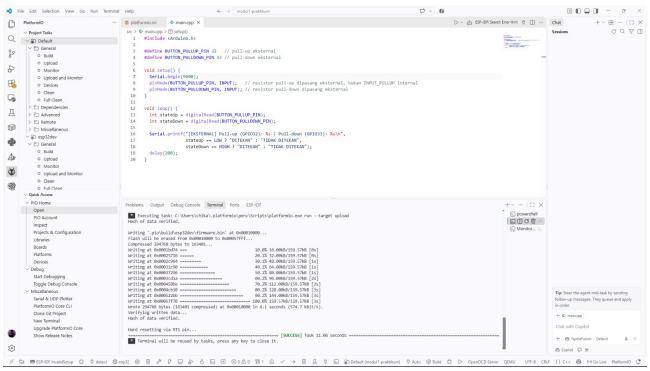
Gambar 2: Rangkaian pull up



Gambar 3: Rangkaian pull down

2. Konfigurasi kembali file platformio.ini sesuai ESP32 Espressif dan pastikan IDE dan ekstensi ter-refresh.

3. Hapus file main.c dan kembali buat file main.cpp sesuai pada modul yang berisi kode untuk pull secara eksternal lalu build dan upload.

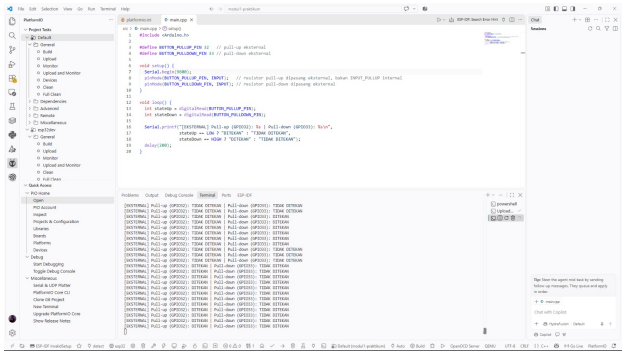


Gambar 4: Kode main.cpp untuk pull-up eksternal

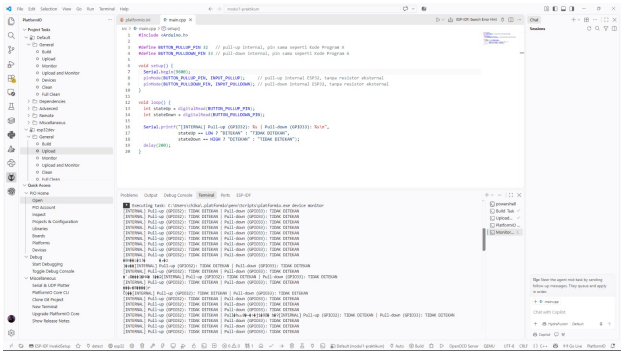
4. Analisa hasil pada serial monitor yang didapatkan dari menekan dan melepas tombol.

5. Ganti isi main.cpp dengan kode untuk pull secara internal sesuai pada modul lalu build dan upload.

6. Analisa kembali hasil yang didapat pada serial monitor.



Gambar 5: Hasil pull up



Gambar 6: Hasil pull down

7. Bandingkan hasil yang didapatkan dari pull eksternal dan internal.

1.5 E. Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Rakit terlebih dahulu ESP32 dengan pushbutton tactile lalu sambungkan ke komputer.

2. Konfigurasi kembali file platformio.ini sesuai ESP32 Espressif dan pastikan IDE dan ekstensi ter-refresh.

3. Konfigurasi 2 kode yang akan digunakan sebagai penghitung, yaitu counterNoDebounce dan counterWithDebounce lalu build dan upload.

4. Tekan tombol satu kali, dan amati hasilnya pada serial monitor.

5. Tekan kembali dengan cara yang berbeda, baik itu cepat atau lambat. Hasil dari counterNoDebounce bisa naik lebih dari 1 untuk satu tekanan, sedangkan counterWithDebounce seharusnya naik tepat 1.

6. Ubah DEBOUNCE DELAY menjadi 10 (10 ms), klik Build dan Upload, lalu tekan tombol lagi.

7. Ubah menjadi 200 (200 ms), klik Build dan Upload, lalu tekan tombol dengan cepat berulang kali.

8. Bandingkan ketiga nilai dari delay yang berbeda. Analisis untung ruginya.

1.6 F. Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Rakit LDR pada GPIO ESP32 dengan resistor 10k lalu sambungkan ke komputer.

2. Konfigurasi kembali file platformio.ini sesuai ESP32 Espressif dan pastikan IDE dan ekstensi ter-refresh.

3. Konfigurasi kode program untuk membaca nilai ADC mentah dan mengubahnya menjadi tegangan lalu build dan upload.

4. Amati hasilnya melalui serial monitor.
5. Tutup LDR dengan tangan, lalu sinari dengan lampu. Lihat nilai mentahnya (0 sampai 4095) berubah.
6. Analisa mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW.

2 Analisis Hasil Percobaan

Pada pengujian keluaran digital (*LED blink*) di percobaan pertama dan kedua, pengendalian pin GPIO diuji menggunakan dua arsitektur mikrokontroler yang berbeda, yakni STM32 Blackpill berbasis *framework* Arduino pada pin ‘PC13‘ serta ESP32 DevKit berbasis *framework* ESP-IDF pada ‘GPIO2‘ dengan interval jeda 500 ms. Hasil pengamatan menunjukkan bahwa kedua papan mikrokontroler mampu mengeksekusi logika keluaran secara akurat sesuai teori, meskipun memiliki karakteristik perangkat keras dan abstraksi pustaka yang berlainan. Pada STM32 Blackpill, LED bawaan papan dirancang dengan konfigurasi *active-LOW* sehingga menyala justru ketika pin diberi logika ‘LOW‘, sedangkan pada ESP32 pengaturan jeda waktu memanfaatkan fungsi ‘vTaskDelay()‘ milik FreeRTOS untuk menunda eksekusi tugas tanpa mengunci siklus prosesor secara penuh. Kelancaran percobaan pertama sempat terhambat oleh kegagalan proses unggah (*upload*) program akibat gangguan *environment* PlatformIO pada laptop awal yang digunakan, yang kemudian berhasil diatasi dengan memindahkan proses kompilasi dan *flashing* ke laptop lain.

Beralih ke pengujian masukan digital pada percobaan ketiga, pembacaan status tombol pada ESP32 diuji melalui konfigurasi *pull-up* (‘GPIO32‘) dan *pull-down* (‘GPIO33‘) dengan membandingkan pemakaian resistor eksternal dan resistor internal (‘INPUTPULLUP‘ serta ‘INPUTPULLDOWN‘). Keluaran yang tampil pada Serial Monitor sepenuhnya selaras dengan teori rangkaian logika, di mana konfigurasi *pull-up* menjaga kondisi awal pin tetap berada pada logika ‘HIGH‘ saat tombol dilepas dan konfigurasi *pull-down* menahannya pada logika ‘LOW‘ sehingga pin terhindar dari kondisi mengambang (*floating*). Dari sisi implementasi rangkaian, pengaktifan resistor internal terbukti jauh lebih praktis karena menyederhanakan pengkabelan pada *breadboard*, sementara penggunaan resistor eksternal tetap unggul dalam memberikan keleluasaan untuk menyesuaikan nilai resistansi maupun memodifikasi skema rangkaian.

Percobaan keempat mengevaluasi kestabilan pembacaan tombol pada ‘GPIO25‘ dengan membandingkan kondisi tanpa *debouncing* terhadap penerapan waktu tunda *debouncing* sebesar 10 ms, 50 ms, dan 200 ms. Tanpa mekanisme penyaringan, satu kali penekanan tombol fisik terbaca berulang kali pada Serial Monitor akibat fenomena *bouncing*, yakni pantulan mekanis kontak logam di dalam sakelar yang memicu osilasi sinyal transien. Ketika algoritma *debouncing* diterapkan, jumlah pembacaan turun secara nyata karena fluktuasi sinyal sesaat tersebut berhasil disaring. Perbandingan ketiga variasi waktu tunda juga memperlihatkan adanya kompromi (*trade-off*) dalam perancangan sistem: interval yang terlalu singkat seperti 10 ms berisiko belum sepenuhnya menyaring sisa pantulan kontak mekanis, sedangkan jeda yang terlalu panjang seperti 200 ms memang memastikan sinyal stabil tetapi menimbulkan latensi yang membuat respons tombol terasa lambat bagi pengguna.

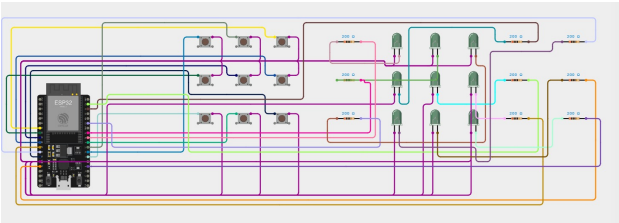
Pada percobaan kelima, kemampuan akuisisi sinyal analog diuji dengan menghubungkan sensor *Light Dependent Resistor* (LDR) ke ‘GPIO34‘ pada ESP32 untuk dibaca melalui fungsi ‘analogRead()‘ dan dikonversi ke satuan tegangan. Nilai mentah ADC yang diperoleh bergerak pada rentang

0 hingga 4095 yang bersesuaian dengan tegangan terukur sekitar 0 V hingga 3,3 V, sebuah hasil yang presisi mengikuti kapasitas resolusi ADC 12-bit milik ESP32 ($2^{12} - 1 = 4095$). Perubahan angka pembacaan ini terjadi karena nilai hambatan LDR berubah mengikuti intensitas cahaya lingkungan sehingga menggeser rasio tegangan pada rangkaian pembagi tegangan (*voltage divider*), dengan arah kenaikan atau penurunan tegangan yang bergantung pada tata letak posisi LDR terhadap resistor tetapnya.

Berdasarkan keseluruhan rangkaian pengujian tersebut, praktikum berjalan dengan baik dan berhasil mencapai tujuan pembelajaran yang ditetapkan. Seluruh data pengamatan pada antarmuka GPIO digital, penyaringan sinyal mekanis tombol, maupun konversi analog-ke-digital (ADC) menunjukkan kesesuaian dengan landasan teori sistem tertanam, sekaligus menegaskan bahwa keberhasilan implementasi di laboratorium tidak hanya bergantung pada ketepatan perancangan rangkaian dan logika program, tetapi juga sangat dipengaruhi oleh kesiapan serta stabilitas perangkat lunak pengembangan seperti PlatformIO pada komputer yang digunakan.

3 Hasil Tugas Modul

3.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3



Gambar 7: Simulasi Rangkaian pada Cirkit

```cpp
// Matrix tombol 3x3 -> LED 3x3, ESP32 DevKitC
// Input aktif LOW: tombol menghubungkan GPIO ke GND.
// Setiap kanal didebounce secara independen; LED berubah hanya setelah
// level tombol stabil selama DEBOUNCE_MS.
const uint8_t buttonPins[9] = {16, 17, 32, 33, 25, 26, 4, 5, 12};
const uint8_t ledPins[9]    = {18, 19, 21, 22, 23, 27, 14, 13, 2};
const unsigned long DEBOUNCE_MS = 30;

bool stablePressed[9];
bool lastRawPressed[9];
unsigned long lastChangeMs[9];

void setup() {
  for (uint8_t i = 0; i < 9; ++i) {
    pinMode(buttonPins[i], INPUT_PULLUP);
    pinMode(ledPins[i], OUTPUT);
    stablePressed[i] = false;
    lastRawPressed[i] = false;
    lastChangeMs[i] = millis();
    digitalWrite(ledPins[i], LOW);
  }
}

void loop() {
  const unsigned long now = millis();
  for (uint8_t i = 0; i < 9; ++i) {
    const bool rawPressed = (digitalRead(buttonPins[i]) == LOW);
    if (rawPressed != lastRawPressed[i]) {
      lastRawPressed[i] = rawPressed;
      lastChangeMs[i] = now;
    }
    if ((rawPressed != stablePressed[i]) &&
        (now - lastChangeMs[i] >= DEBOUNCE_MS)) {
      stablePressed[i] = rawPressed;
      digitalWrite(ledPins[i], stablePressed[i] ? HIGH : LOW);
    }
  }
}
```

Gambar 8: Source Code

3.2 Pertanyaan Analisis

1. Meskipun pengguna umumnya hanya menekan satu tombol dalam satu waktu, proses debouncing tetap wajib diterapkan karena sumber masalahnya bukan berasal dari penekanan banyak tombol secara bersamaan, melainkan dari karakteristik fisik kontak mekanik pada tombol itu sendiri. Sebagaimana dijelaskan pada bagian C.5 Debouncing di modul praktikum, pelat logam di dalam pushbutton mekanik mengalami getaran fisik atau pantulan sesaat ketika kontak pertama kali bersentuhan maupun saat dilepaskan. Getaran mikro ini menghasilkan beberapa kali transisi sinyal logika HIGH dan LOW secara beruntun dalam rentang waktu yang sangat singkat (umumnya 20 hingga 50 milidetik).

2. Pada perancangan pembacaan matrix pushbutton, pin kolom mikrokontroler umumnya dikonfigurasi memakai mode pull-up (baik secara internal melalui INPUT PULLUP maupun secara eksternal dengan resistor 10 kΩ menuju jalur 3,3 V). Dalam konfigurasi ini, ketika tidak ada tombol yang ditekan, jalur kolom tidak terhubung ke jalur baris yang berlogika rendah (GND), sehingga resistor pull-up menahan tegangan pin kolom tetap berada pada kondisi logika HIGH (sekitar 3,3 V). Logika pada pin kolom tersebut baru akan berubah menjadi LOW apabila sebuah tombol ditekan dan menyambungkan pin kolom ke pin baris yang sedang dipindai pada level tegangan rendah. Pin input mikrokontroler tidak boleh dibiarkan dalam keadaan mengambang (floating) tanpa acuan tegangan yang pasti karena impedansi masukan gerbang digital sangat tinggi. Jika pin dibiarkan floating, jalur tembaga pada papan rangkaian maupun kabel penghubung dapat bertindak layaknya antena yang menangkap derau elektromagnetik (noise) dari lingkungan sekitar maupun kapasitansi liar antarkabel. Derau tersebut menyebabkan tegangan pada pin berfluktuasi secara acak di antara ambang batas logika rendah dan tinggi, sehingga mikrokontroler akan mendeteksi penekanan tombol palsu meskipun tidak ada kontak fisik yang terjadi.

3. Perbedaan mendasar antara pembacaan digital input dan analog input terletak pada cara mikrokontroler mengkuantisasi dan menafsirkan level tegangan listrik yang masuk ke pin GPIO. Pada mode digital input (seperti pembacaan tombol), rangkaian komparator internal di dalam pin GPIO hanya membandingkan tegangan masukan terhadap ambang batas (threshold) tertentu untuk mengelompokkannya ke dalam dua keadaan biner diskret, yaitu logika LOW (0) atau HIGH (1). Seluruh variasi tegangan yang berada di bawah batas rendah atau di atas batas tinggi akan disederhanakan menjadi dua nilai tersebut saja, sehingga informasi mengenai besarnya perubahan tegangan di antaranya hilang. Sebaliknya, ketika salah satu tombol digantikan oleh Light Dependent Resistor (LDR) yang dirangkai sebagai pembagi tegangan dan dibaca melalui Analog-to-Digital Converter (ADC), tegangan masukan yang berubah secara kontinu seiring intensitas cahaya tidak lagi dipangkas menjadi dua kutub biner. Modul ADC pada ESP32 memiliki resolusi 12-bit, yang berarti rentang tegangan kerja dari 0 V hingga 3,3 V dibagi ke dalam 2^{12} atau 4.096 tingkatan kuantisasi diskret (bernilai 0 sampai 4095) dengan langkah ketelitian sekitar 0,8 milivolt per tingkat. Karena kapasitas pencacahan yang jauh lebih halus inilah ADC mampu merepresentasikan ribuan gradasi intensitas cahaya secara rinci, tidak terbatas pada kondisi terang atau gelap semata.

4 Kesimpulan

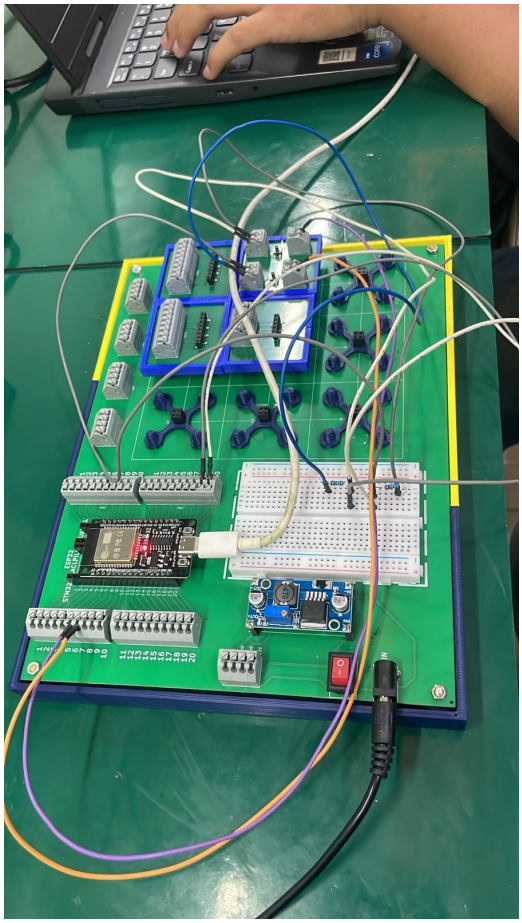
Praktikum ini berhasil mencapai tujuannya dalam menguji antarmuka masukan dan keluaran dasar pada mikrokontroler STM32 Blackpill serta ESP32 DevKit. Pada pengujian keluaran digital (*LED blink*) dengan jeda 500 ms, kedua papan mikrokontroler tersebut bekerja sesuai teori meskipun memakai *framework* dan fungsi waktu yang berbeda, yaitu Arduino pada pin ‘PC13‘ yang bersifat *active-LOW* serta ESP-IDF pada ‘GPIO2‘ dengan ‘vTaskDelay()‘ milik FreeRTOS. Pengujian masukan tombol pada ‘GPIO32‘ dan ‘GPIO33‘ juga menunjukkan hasil yang sesuai dengan teori rangkaian logika, sebab konfigurasi *pull-up* mampu menjaga kondisi awal pin tetap pada logika ‘HIGH‘ dan konfigurasi *pull-down* menahannya pada logika ‘LOW‘. Selain itu, penerapan metode *debouncing* pada ‘GPIO25‘ terbukti dapat menyaring pantulan mekanis kontak tombol, dengan variasi waktu tun-

da 50 ms memberikan keseimbangan terbaik antara kestabilan pembacaan dan kecepatan respons dibandingkan jeda 10 ms maupun 200 ms.

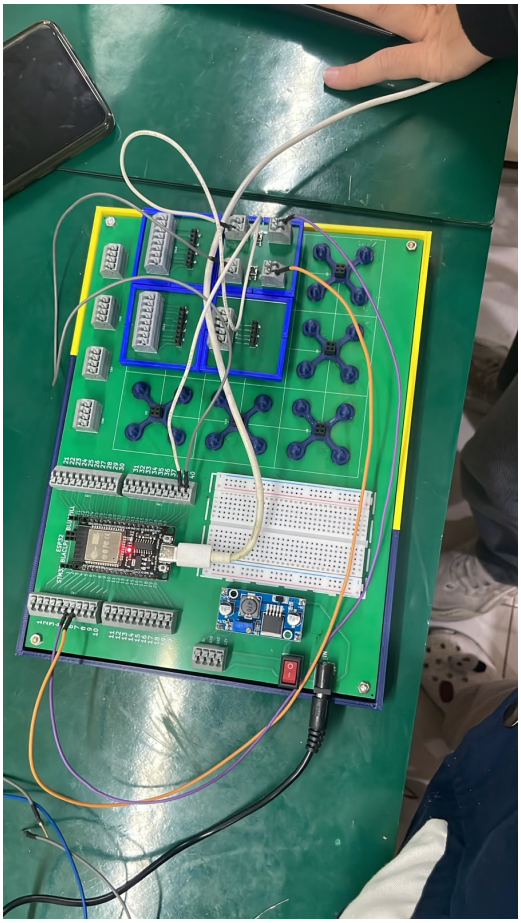
Pada pengujian sinyal analog menggunakan sensor *Light Dependent Resistor* (LDR) yang terhubung ke ‘GPIO34‘, nilai ADC yang terbaca pada rentang 0 hingga 4095 atau setara tegangan 0 V sampai 3,3 V telah sesuai dengan karakteristik resolusi ADC 12-bit milik ESP32. Perubahan nilai pembacaan itu terjadi seiring perubahan hambatan LDR akibat variasi intensitas cahaya pada rangkaian pembagi tegangan. Selama pelaksanaan praktikum tidak ditemukan hasil pengukuran yang menyimpang dari teori, meskipun proses pengujian sempat terhambat oleh kegagalan unggah (*upload*) program akibat gangguan *environment* PlatformIO pada laptop awal sebelum akhirnya dialihkan ke laptop lain. Dari kejadian tersebut, praktikan memperoleh pembelajaran bahwa kelancaran implementasi sistem tertanam turut bergantung pada kesiapan serta kestabilan perangkat lunak pengembangan di samping ketepatan perancangan rangkaian dan kode program.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 9



Gambar 10